

Data engineering challenge

Energy Data Pipeline

Presented by: Dhouha HMEM



Content

- 01 Project Overview
- 02 Project Architecture
- 03 Project Execution
- 04 Results
- 05 Conclusion
- 06 Possible Improvements





Project Overview

Problem Statement

What problem are we solving?

Energy market data is:

- High-frequency (15-min / hourly)
- Semi-structured (JSON)
- Spread across multiple endpoints

Goal:

- Build a reliable, configurable pipeline
- Transform raw API data into daily analytical datasets



Project Objectives

What the pipeline must achieve

- Ingest public power production data (Germany)
- Ingest electricity price data (DE-LU)
- Support historical backfill
- Produce daily aggregated metrics
- Keep configuration outside the code
- Use Delta Lake for reliability



Solution

The implementation of a local data pipeline using PySpark and Delta Lake to ingest, transform, and analyze electricity production and price data from the public API: <https://api.energy-charts.info>

➔ batch, local, reproducible pipeline

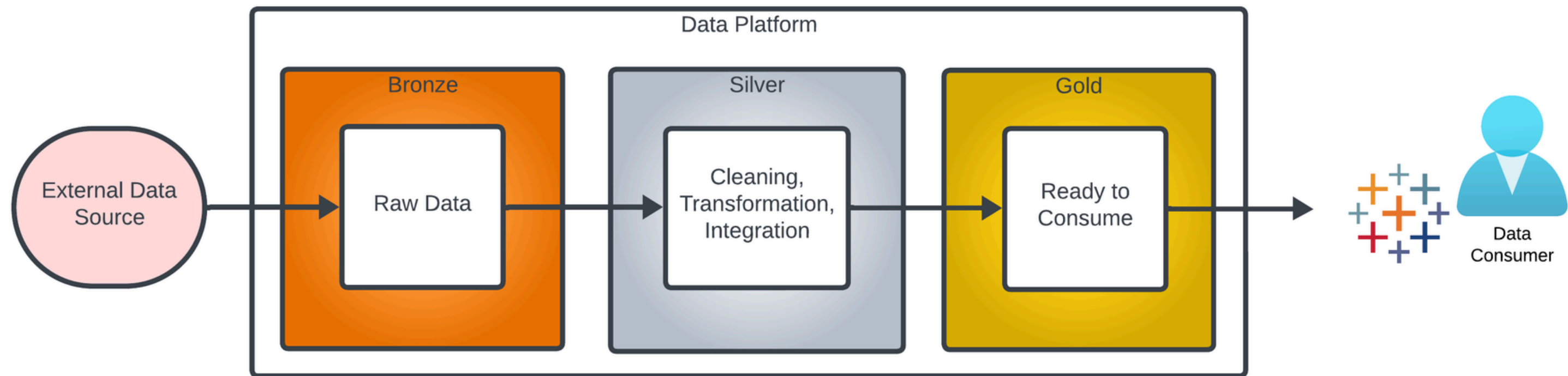


Project Architecture

Medallion Architecture

The pipeline follows the Medallion Architecture:

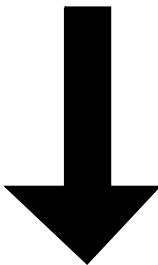
- Bronze: raw ingested data (JSON)
- Silver: clean, typed, normalized, time-aligned data
- Gold: aggregated, analytics-ready data



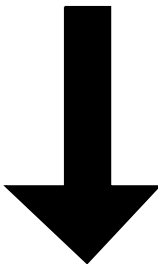


Medallion Architecture

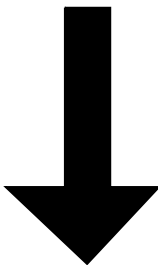
Energy-Charts API



Bronze Layer (raw API responses)



Silver Layer (exploded and structured data)



Gold Layer (daily aggregates & analytics-ready)

Project Structure

```
energy-data-pipeline/
├── config/
│   └── config.yaml           #Pipeline configuration
├── src/
│   ├── __init__.py          #Makes the folder a Python package
│   ├── ingestion/           #Bronze ingestion logic
│   │   ├── power_ingestion.py
│   │   └── price_ingestion.py
│   ├── transformations/
│   │   ├── __init__.py
│   │   ├── bronze_to_silver.py
│   │   └── silver_to_gold.py
│   └── utils/
│       ├── __init__.py
│       ├── api_client.py     # Energy-Charts API client
│       ├── dates.py          # Date & backfill logic
│       ├── paths.py          # Medallion layer paths
│       └── spark_session.py  # Spark + Delta setup
├── data/
│   ├── bronze/
│   ├── silver/
│   └── gold/
├── main.py                   # Pipeline entry point
├── requirements.txt
└── README.md
```

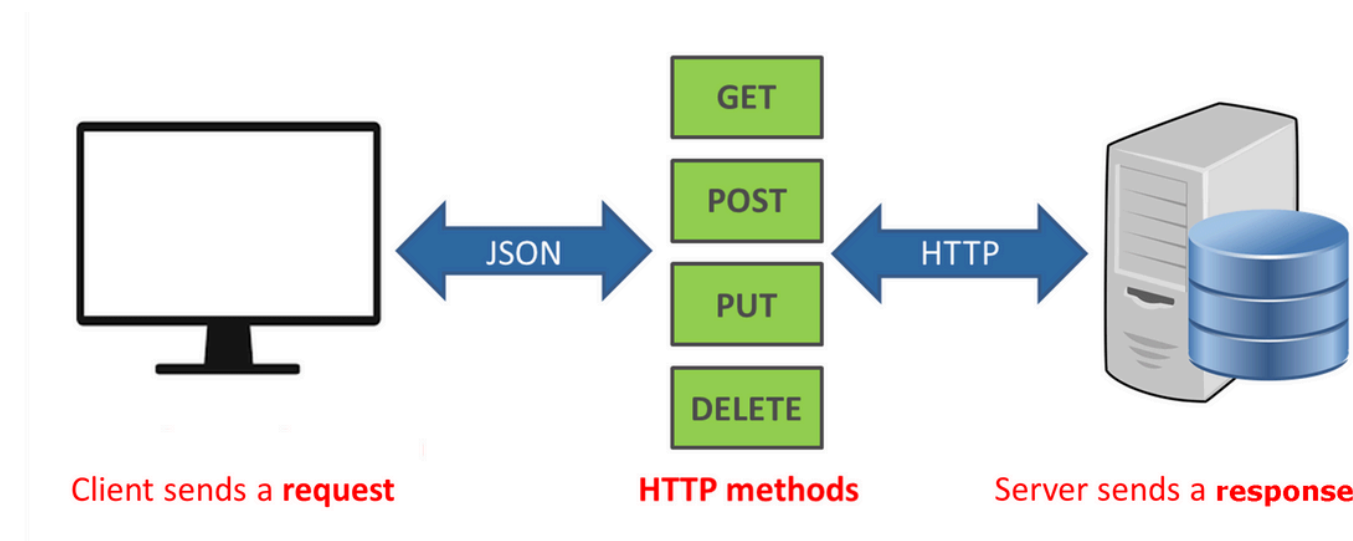


Project Execution

Technologies Used



- Visual Studio Code : project organization
- Python : orchestration & logic
- PySpark : scalable transformations
- Delta Lake : ACID tables, reliability
- YAML : flexible configuration
- Requests : API communication



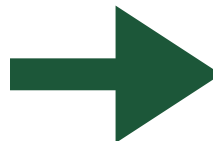


Data source

 <https://api.energy-charts.info>: This is an open public API

Dataset A: Public power production

- Net electricity production by production type (wind, solar, coal, etc.)
- Resolution: 15 minutes
- Country: Germany

 Trend of daily public net electricity production in Germany for each production type





Data source

Dataset B : Electricity prices (DE-LU price zone)

- Day-ahead electricity prices
- Price zone: DE-LU
- Resolution:
 - Hourly before 01-10-2025
 - 15-minute after that

 Correlation analysis of daily price against net power for offshore wind





Phase 1: Data Ingestion (Bronze Layer)

Power production (public power)

- GET https://api.energy-charts.info/public_power
- Typical parameters:
 - country=de
 - start=YYYY-MM-DD
 - end=YYYY-MM-DD
- Returns:
 - timestamps
 - production values per energy source

Electricity prices (DE-LU)

- GET <https://api.energy-charts.info/price>
 - Typical parameters:
 - bzn=DE-LU
 - start=YYYY-MM-DD
 - end=YYYY-MM-DD
 - Returns:
 - timestamps
 - price values (EUR/MWh)
- 

Phase 1: Data Ingestion (Bronze Layer)

when start to run the pipeline, main.py does this:

1. Reads config.yaml (base_url + dates)
2. Builds a list of dates (e.g., 2025-01-01 ... 2025-01-07) (dates.py)
3. For each date, calls the API twice:
 - /public_power (Germany power production)
 - /price (DE-LU price)
4. Saves the raw response into Bronze Delta as a JSON string

```
ingestion_frequency: "daily"
backfill:
  enabled: true
  start_date: "2025-01-01"
  end_date: "2025-01-07"
```

```
base_url: "https://api.energy-charts.info"
timeout_seconds: 60
user_agent: "energy-data-pipeline/1.0"
```



Phase 1: Data Ingestion (Bronze Layer)

Public power payload_json example :

```
{  
  "unix_seconds": [...],  
  "production_types": [  
    {"name": "Wind offshore", "data": [...]},  
    {"name": "Biomass", "data": [...]}  
  ], ...  
}
```

Price payload_json example:

```
{  
  "unix_seconds": [...],  
  "price": [...],  
  "unit": "EUR / MWh"  
}
```





Phase 2: Data Transformation (Silver Layer)

- Parse JSON payload
- Explode arrays into rows
- Convert Unix to timestamps
- Create clean, structured schema

Result:

- One row per timestamp + metric(One row = one observation)
 - Ready for aggregation
- 



Phase 2: Data Transformation (Silver Layer)

- Extract timestamps and metrics

Example:

```
"unix_seconds": [1704067200, 1704068100]
```

```
"price": [95.2, 97.1]
```

- Zip timestamps with prices: Use `arrays_zip` to match each timestamp with its price

Example:

```
(unix_seconds=1704067200, price=95.2)
```

```
(unix_seconds=1704068100, price=97.1)
```





Phase 2: Data Transformation (Silver Layer)

- Convert timestamps to Spark timestamp type:

Convert UNIX seconds → readable timestamps

Example:

1704067200 → 2025-01-01 00:00:00

- Clean the data: Remove rows with missing timestamps and missing prices

Example

NULL timestamp ❌ → dropped

Valid timestamp ✅ → kept





Phase 2: Data Transformation (Silver Layer)

- Write clean, structured Silver tables: Silver tables are analytics-ready
 - One row per measurement
 - Used as input for Gold aggregations

Example:

market	timestamp	price_eur_mwh
DE-LU	2025-01-01 00:00:00	95.2





Phase 3: Aggregation (Gold Layer)

Daily production:

- To get daily total production, we sum all 15-min values of the same day.

Example (Wind offshore, 2025-01-01):

Silver rows: (...96 points/day).

00:00 → 120

00:15 → 130

00:30 → 140

Gold calculation:

$\text{daily_net_production} = 120 + 130 + 140 + \dots = \text{total for the day}$





Phase 3: Aggregation (Gold Layer)

Daily price:

- To get one daily price number, we take the average of all prices in that day.

Example (2025-01-01):(...24 points/day if hourly)

Silver prices:

- 01:00 → 80
- 02:00 → 100
- 03:00 → 90

Gold calculation:

- $\text{avg_price} = (80 + 100 + 90 + \dots) / 24 = \text{daily average}$
- 



Phase 3: Aggregation (Gold Layer)

3 outputs:

A) power_daily_by_type : (Daily net electricity production per production type)

- Group by (date, production_type)
- SUM(value) → daily total production
- Output columns:
date | production_type | daily_net_production

B) price_daily: Daily average electricity price for DE-LU

- Group by date
 - AVG(price_eur_mwh) → daily average price
 - Output columns:
date | avg_price_eur_mwh
- 

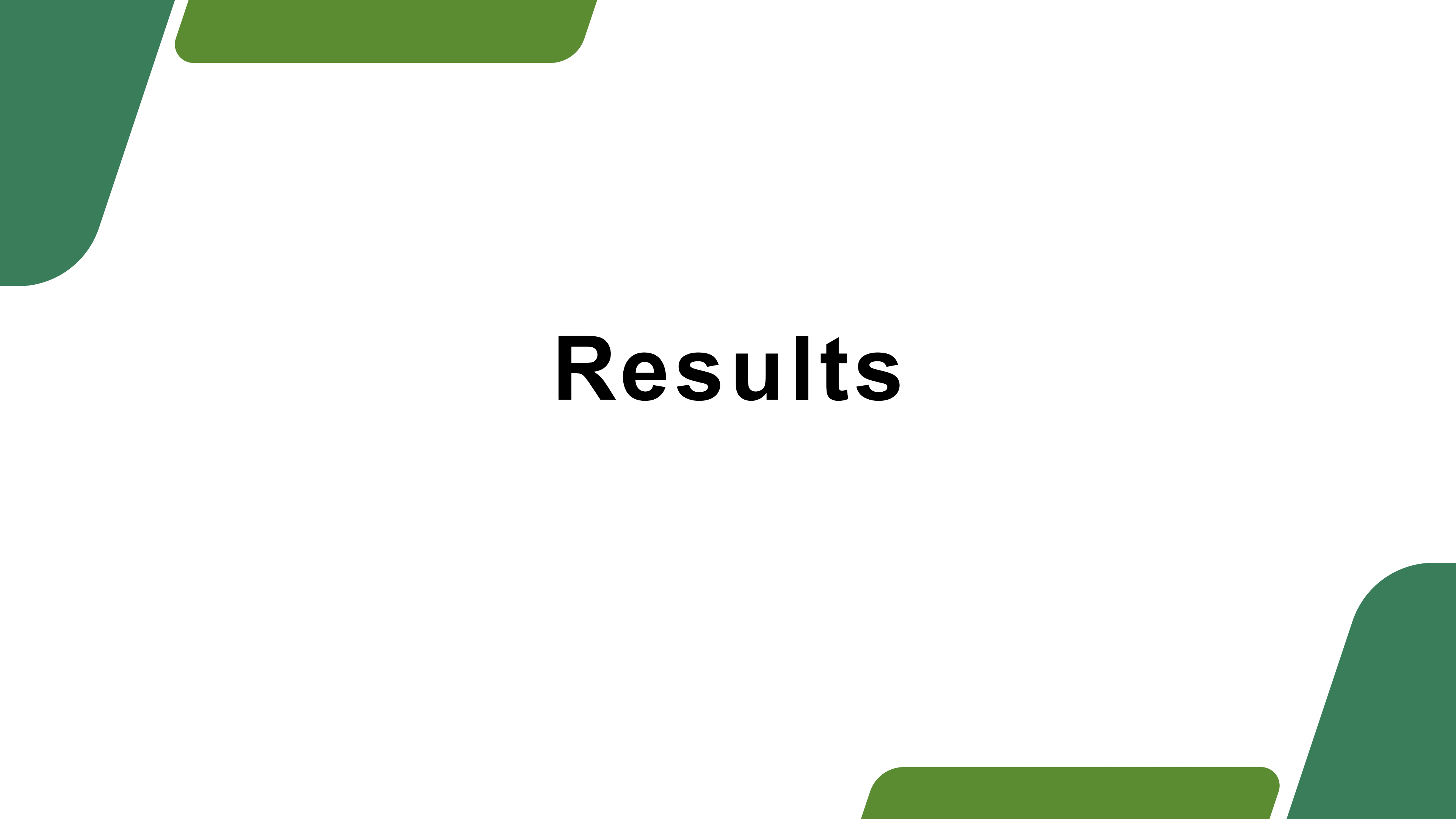


Phase 3: Aggregation (Gold Layer)

3 outputs:

C) price_daily: Correlation analysis of daily price against offshore wind production

- Convert timestamp → date
 - Group by date
 - AVG(price_eur_mwh) → daily average price
 - Output columns:
date | avg_price_eur_mwh
- 



Results

Results

```
✓ Spark started | Delta enabled = True
→ Ingesting data from 2025-01-01 to 2025-01-07 (7 day(s))
✓ Bronze public power (DELTA) ingested for 7 day(s)
✓ Bronze price (DELTA) ingested for 7 day(s)
✓ Silver public power table created (DELTA)
✓ Silver price table created (DELTA)
✓ Gold daily power production created (DELTA)
✓ Gold daily price table created (DELTA)
✓ Gold power-price join table created (DELTA)
```

✓ Pipeline finished successfully

```
power_bronze count = 7
price_bronze count = 7
power_silver count = 14112
price_silver count = 168
power_gold count = 147
price_gold count = 7
power_price_gold count = 7
```

```
Distinct production_type (sample):
+-----+
|production_type|
+-----+
|Biomass|
|Cross border electricity trading|
|Fossil brown coal / lignite|
|Fossil coal-derived gas|
|Fossil gas|
|Fossil hard coal|
|Fossil oil|
|Geothermal|
|Hydro Run-of-River|
|Hydro pumped storage|
|Hydro pumped storage consumption|
|Hydro water reservoir|
|Load|
|Others|
|Renewable share of generation|
|Renewable share of load|
|Residual load|
|Solar|
|Waste|
|Wind offshore|
|Wind onshore|
+-----+
```

Results

Bronze layer:

- ingest 7 days
- Each API response = 1 JSON payload per day(one row per API call per day)
==> Bronze count = 7

Silver layer:

- Public power data resolution: 15 minutes
- Measurements per day: $24 \text{ hours} \times 4 = 96$
- Production types: ~21
- Days ingested: 7

$7 \text{ days} \times 96 \text{ intervals} \times \sim 21 \text{ production types} \approx 14112 \text{ rows}$

Price data resolution:

- Hourly (for our time range)
- $24 \text{ hours} \times 7 \text{ days} = 168$

Results

Gold layer:

Gold power aggregates, Grouped by:

- date
- production_type
- So: 7 days $\times$ ~21 production types $\approx$ 147 rows

Gold price aggregates, Grouped by:

- date
- Metric: daily average price
- So: 7 days $\rightarrow$ 7 rows

Gold join table:

- This table is filtered to:
 - production_type = "Wind offshore"
 - joined on date
- So: 1 row per day and 7 days total

Results

power_gold sample:

date	production_type	daily_net_production
2025-01-01	Hydro pumped storage consumption	-277374.69999999984
2025-01-01	Cross border electricity trading	-1175918.2999999998
2025-01-01	Hydro Run-of-River	165770.89999999997
2025-01-01	Biomass	382260.30000000005
2025-01-01	Fossil brown coal / lignite	316412.3
2025-01-01	Fossil hard coal	216356.3
2025-01-01	Fossil oil	36690.20000000001
2025-01-01	Fossil coal-derived gas	49087.200000000004
2025-01-01	Fossil gas	332903.09999999986
2025-01-01	Geothermal	2088.8999999999999
2025-01-01	Hydro water reservoir	6845.3
2025-01-01	Hydro pumped storage	20687.5
2025-01-01	Others	19598.2
2025-01-01	Waste	95757.70000000001
2025-01-01	Wind offshore	272729.80000000005
2025-01-01	Wind onshore	3521231.9
2025-01-01	Solar	215798.60000000001
2025-01-01	Load	4721965.599999999
2025-01-01	Residual load	712205.7999999997
2025-01-01	Renewable share of load	9358.3
2025-01-01	Renewable share of generation	7811.399999999998
2025-01-02	Hydro pumped storage consumption	-57796.399999999994
2025-01-02	Cross border electricity trading	-517438.6999999999
2025-01-02	Hydro Run-of-River	167475.69999999992

price_gold sample:

date	avg_price_eur_mwh
2025-01-01	0.9545833333333335
2025-01-02	92.90958333333333
2025-01-03	89.94083333333333
2025-01-04	120.79333333333334
2025-01-05	72.23750000000001
2025-01-06	25.163750000000007
2025-01-07	58.290416666666665

power_price_gold sample:

date	offshore_wind_daily	avg_price_eur_mwh
2025-01-01	272729.80000000005	0.9545833333333335
2025-01-02	525144.90000000001	92.90958333333333
2025-01-03	387397.89999999999	89.94083333333333
2025-01-04	225555.80000000002	120.79333333333334
2025-01-05	429884.20000000001	72.23750000000001
2025-01-06	439999.10000000015	25.163750000000007
2025-01-07	371129.00000000001	58.290416666666665



Conclusion



```

config.yaml
|
|  start_date, end_date
v
dates.py  --->  ["2025-01-01", "2025-01-02", ...]
|
v
main.py  (for each day)
|
+--> api_client.get_public_power(country="de", start=d, end=d)
|
|  v
|  https://api.energy-charts.info/public_power?country=de&start=d&end=d
|
+--> api_client.get_price(bzn="DE-LU", start=d, end=d)
|
|  v
|  https://api.energy-charts.info/price?bzn=DE-LU&start=d&end=d

(API returns JSON payloads)
|
v
Bronze (Delta): 1 row per day, raw JSON string
|
v
Silver (Delta): exploded time series rows (timestamp + value)
|
v
Gold (Delta): daily aggregations + join for correlation

```



What this project demonstrates:

- Data engineering fundamentals
- Clean architecture
- Thoughtful design decisions
- Production-oriented mindset

Limitations:

- Local execution only
- No scheduling yet
- Limited to daily ingestion



Possible Improvements

- **Incremental ingestion:** Instead of overwriting data on every run, the pipeline would ingest only new time periods, improving efficiency and avoiding unnecessary reprocessing.
- **Add unit tests:** Introducing automated tests to ensure that ingestion, transformation, and aggregation logic continue to work correctly.
- **Support hourly ingestion:** Extending the pipeline to handle hourly ingestion to allow more frequent data updates and finer-grained analysis when required.
- **Move to cloud storage:** Migrating data storage from local disk to cloud storage to improve scalability, durability, and readiness for production environments.



THANK YOU
FOR YOUR
ATTENTION

